



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA
DIPARTIMENTO DI ECCELLENZA



CORSO DI LAUREA IN INFORMATICA

Tecnologie Software per il Web

HTML Parte 2

PROF. ATTILIO DELLA GRECA

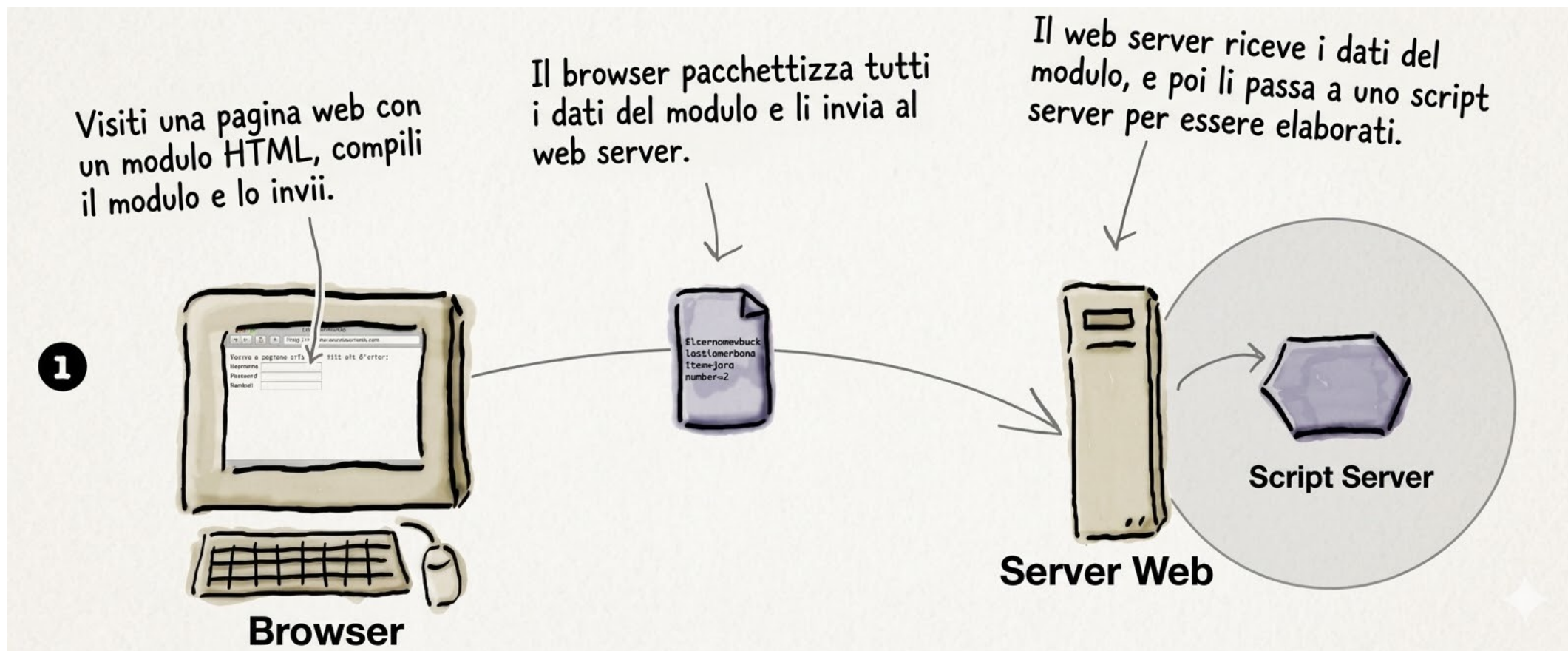
a.a. 2025-2026

Form

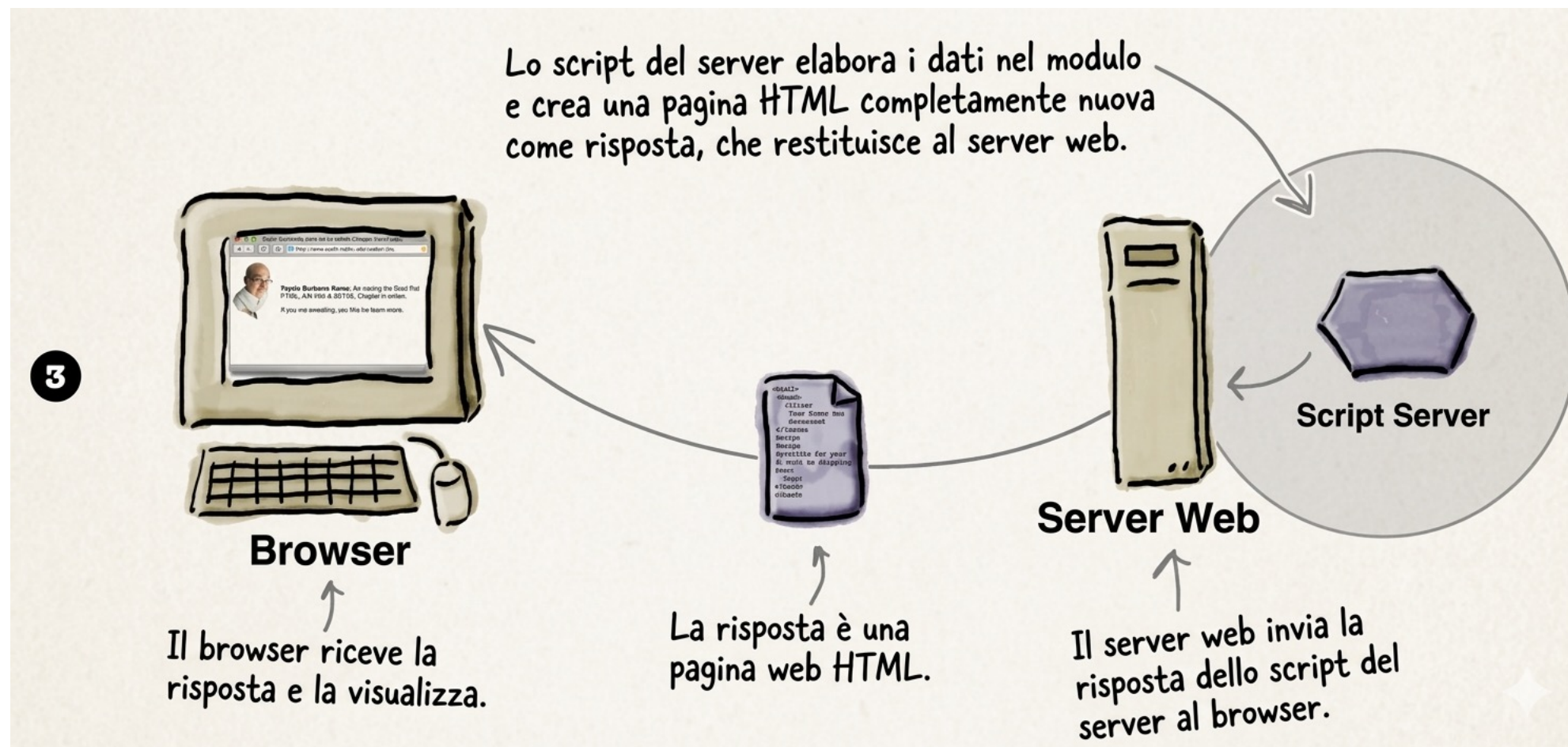
I form (moduli) sono l'unico meccanismo nativo HTML per raccogliere dati dall'utente e inviarli al server. L'elemento `<form>` definisce un'area interattiva che contiene campi di input, menu, checkbox, pulsanti e altri controlli. Quando l'utente preme "Invia", i dati vengono raccolti e spediti all'URL specificato.

- Bottoni di azione
- Checkbox (caselle di spunta)
- Radio Button (bottoni mutuamente esclusivi)
- Liste di selezione (lista di opzioni)
- Caselle di inserimento di testo
- *Oggetti nascosti (elementi valorizzati ma invisibili)*
- Selezione file
- ...

Forms



Forms



<form> tag

Il tag **<form>** racchiude tutti gli elementi del modulo (è un elemento di tipo blocco)

action URL di destinazione per l'invio dei dati. Se omissso, i dati vengono inviati alla pagina corrente.

- Può essere un percorso relativo (/api/submit) o assoluto (https://...).

method (specifica il modo in cui i dati vengono inviati)

- **GET** — Usato per ricerche e filtri.
- **POST** — Usato per registrazioni, login, dati sensibili, modifiche.

enctype specifica il content type usato per la codifica (encoding) dei dati contenuti nel form

novalidate Attributo booleano che disabilita la validazione nativa del browser. Utile quando la validazione è gestita interamente con JavaScript.

<form> tag

Il tag **<form>** racchiude tutti gli elementi del modulo (è un elemento di tipo blocco)

```
<body>

<form action="http://wickedlysmart.com/hfhtmlcss/contest.php"
      method="POST">

  (A) <p>Scrivi semplicemente il tuo nome (e fai clic su
      Invia) per partecipare al concorso: <br>

  (B)      Nome: <input type="text" name="firstname" value=""> <br>
  (C)      Cognome: <input type="text" name="lastname" value=""> <br>
  (D)      <input type="submit">

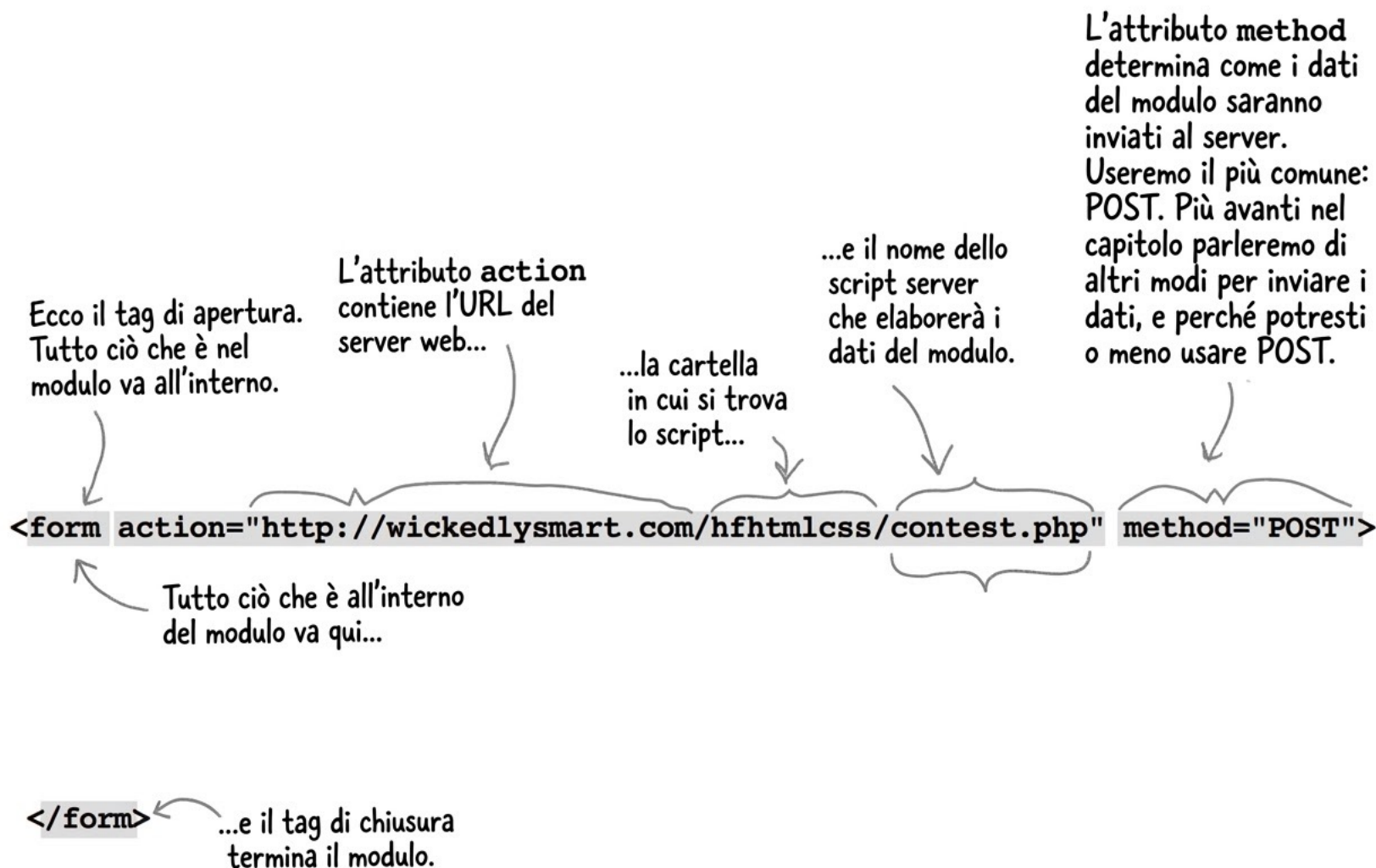
      </p>
  </form>

</body>
</html>
```

← Abbiamo l'elemento `<form>` vero e proprio...

← ...e un gruppo di elementi annidati al suo interno.

<form> tag



<form> tag

method="GET"

I dati vengono accodati all'URL come query string:
/search?q=html&lang=it

Caratteristiche:

- Dati visibili nell'URL e nella cronologia del browser
- Lunghezza limitata (~2048 caratteri)
- La richiesta può essere salvata nei bookmark
- Può essere messa in cache dal browser
- Idempotente: ripetere non cambia lo stato

Usi tipici: ricerche, filtri, paginazione, condivisione link con parametri.

method="POST"

I dati vengono inviati nel corpo (body) della richiesta HTTP, non appaiono nell'URL.

Caratteristiche:

- Dati NON visibili nell'URL
- Nessun limite di dimensione pratico
- Non può essere salvato nei bookmark
- Non viene messo in cache
- Necessario per upload di file (enctype multipart)

Usi tipici: registrazioni, login, invio dati sensibili, creazione/modifica risorse, upload file.

<label> tag

<label> associa etichetta al suo campo. Click sulla label → focus sul campo. Senza label, screen reader non annuncia a cosa si riferisce l'input.

Metodo 1: for/id (raccomandato)

```
<label for="email">Email:</label>
<input type="email" id="email"
      name="email">
```

L'attributo for del <label> deve corrispondere esattamente all'id dell'<input>. L'etichetta e il campo possono essere in qualsiasi posizione nel DOM.

Metodo 2: wrapping (implicito)

```
<label>
  Email:
  <input type="email" name="email">
</label>
```

L'<input> è figlio diretto del <label>. Non serve for/id perché l'associazione è implicita. Meno flessibile per il layout.

Migliora la SEO: i motori di ricerca comprendono meglio la struttura del form.

type="text"

È il tipo di input predefinito e il più semplice. Accetta qualsiasi stringa di testo. Se l'attributo type è omissso o ha un valore non riconosciuto, il browser utilizza automaticamente text come fallback.

```
<input type="text" id="nome" name="nome" placeholder="Mario Rossi"
      maxlength="100" minlength="2" required autocomplete="name" spellcheck="true">
```

placeholder

Testo suggerimento mostrato quando il campo è vuoto. Scompare quando l'utente inizia a digitare. NON sostituisce la <label>: gli screen reader non lo leggono in modo affidabile e scompare appena si digita.

maxlength / minlength

Numero massimo e minimo di caratteri. maxlength impedisce fisicamente di digitare oltre il limite. minlength mostra errore alla validazione se il testo è troppo corto.

size

Larghezza visiva del campo in numero di caratteri. Non limita l'input, solo l'aspetto. Meglio usare CSS width per il dimensionamento.

value

Valore iniziale precompilato del campo. L'utente può modificarlo. Nel form inviato, il valore attuale (non quello iniziale) viene spedito.

autocomplete

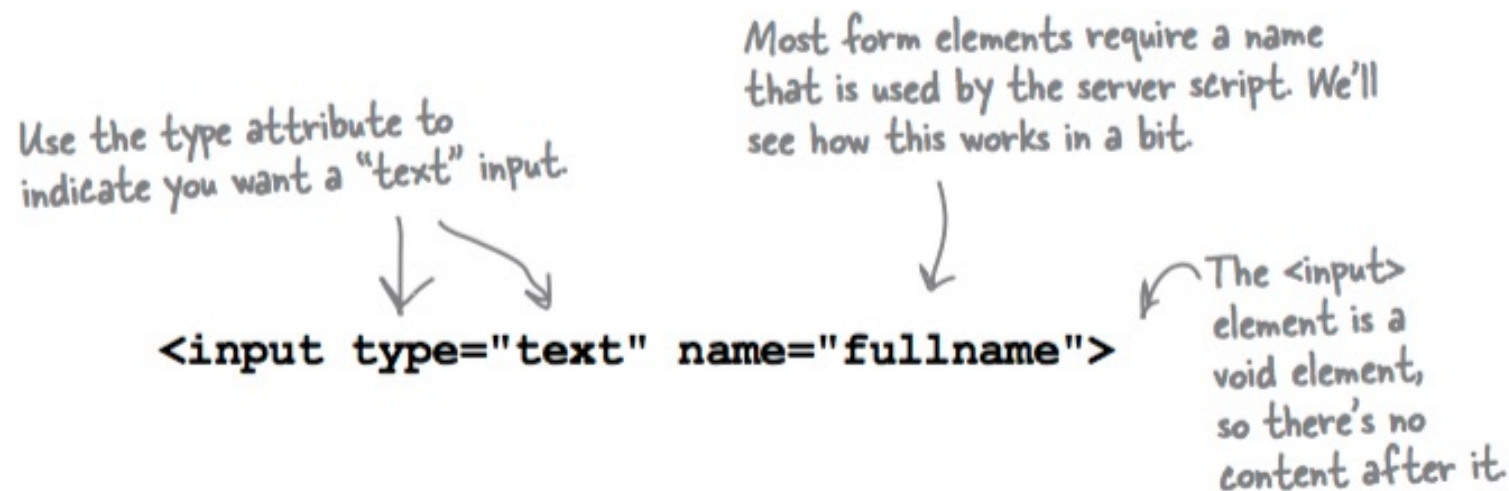
Suggerisce al browser quali dati autocompletare: "name", "email", "tel", "address-line1", "cc-number", "off" per disabilitare.

spellcheck

true/false — attiva o disattiva il controllo ortografico del browser. Utile per campi dove il testo libero è significativo.

type="text"

È il tipo di input predefinito e il più semplice. Accetta qualsiasi stringa di testo. Se l'attributo type è omesso o ha un valore non riconosciuto, il browser utilizza automaticamente text come fallback.



```
<form action="http://site.com/bin/adduser" method="post">
  <p>
    Nome: <input type="text" name="firstname">
  </p>
</form>
```

Nome:

I pulsanti: submit, reset e button

Esistono due modi diversi per creare pulsanti all'interno di un form: l'elemento **<input>** con i tipi submit, reset e button, e l'elemento **<button>** con gli stessi tipi.

<button>

```
<button type="button">Apply now</button>
```

```
<button type="submit">Submit</button>
```

```
<button type="reset">Reset</button>
```

<input>

```
<input type="button" value="Apply now" />
```

```
<input type="submit" value="Submit" />
```

```
<input type="reset" value="Reset" />
```

Button:

Input:

I pulsanti: submit, reset e button

Esistono due modi diversi per creare pulsanti all'interno di un form: l'elemento **<input>** con i tipi submit, reset e button, e l'elemento **<button>** con gli stessi tipi.

L'elemento **<input>** è un elemento void, il che significa che non ha un tag di chiusura e non può contenere nulla al suo interno. Quando viene usato come pulsante, il testo visualizzato è determinato esclusivamente dall'attributo value

<input>

```
<input type="button" value="Apply now" />
```

```
<input type="submit" value="Submit" />
```

```
<input type="reset" value="Reset" />
```

```
<form action="http://site.com/bin/adduser" method="post">  
  <input type="submit" value="Conferma">&nbsp;&nbsp;   
  <input type="reset" value="Azzera">  
</form>
```

I pulsanti: submit, reset e button

Esistono due modi diversi per creare pulsanti all'interno di un form: l'elemento **<input>** con i tipi submit, reset e button, e l'elemento **<button>** con gli stessi tipi.

L'elemento **<button>** è un elemento con tag di apertura e chiusura, il che significa che può contenere qualsiasi contenuto HTML al suo interno: testo, immagini, icone SVG praticamente qualsiasi elemento inline.

```
<button type="reset">
```

```
 Cancella tutto
```

```
</button>
```

```
<form action="http://site.com/bin/adduser" method="post">
  <button type="button">Generico</button>&nbsp;
  <button type="reset"><i>Azzera</i></button>&nbsp;
  <button type="submit"><b>Invia</b></button>
</form>
```

I pulsanti: submit, reset e button

type="submit"

Invia il form all'URL specificato in action con il metodo method.
Attiva la validazione nativa prima dell'invio.

Due sintassi:

`<button type="submit">` (può contenere HTML)

`<input type="submit">` (solo testo nel value).

type="reset"

Riporta tutti i campi del form ai loro valori iniziali (quelli nel codice HTML, non vuoti).

Quasi mai utilizzato nei form moderni perché l'utente raramente vuole cancellare tutto ciò che ha compilato.

Il reset accidentale è un problema di UX molto frustrante

type="button"

Pulsante generico senza alcun comportamento predefinito.

Non invia il form, non resetta i campi. Tutto il suo comportamento è definito tramite JavaScript.

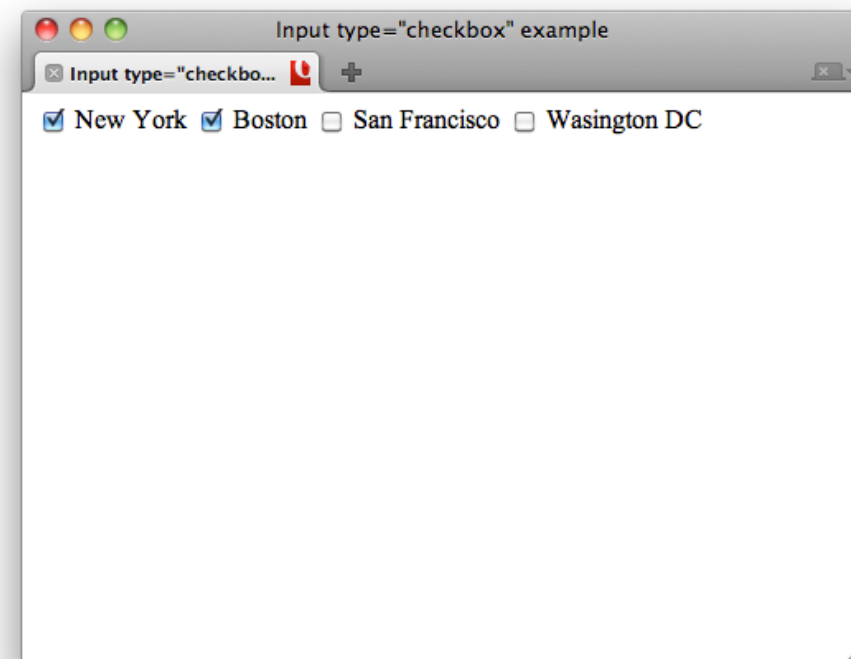
Usato per azioni che non richiedono l'invio dei dati: apertura modali, calcoli in tempo reale, toggle di elementi ecc.

type="checkbox"

crea una casella di controllo che può essere attivata o disattivata indipendentemente dalle altre. Ogni checkbox è un campo autonomo

- Ogni checkbox con lo stesso name invia valori multipli al server
- checked rende la casella preselezionata
- Il value viene inviato solo se la casella è spuntata
- L'area cliccabile è molto piccola: la <label> è essenziale per ingrandirla

```
<label><input type="checkbox" name="ny"> New York</label>  
<label><input type="checkbox" name="bos"> Boston</label>  
<label><input type="checkbox" name="sfo"> San Francisco</label>  
<label><input type="checkbox" name="dc"> Wasington DC</label>
```

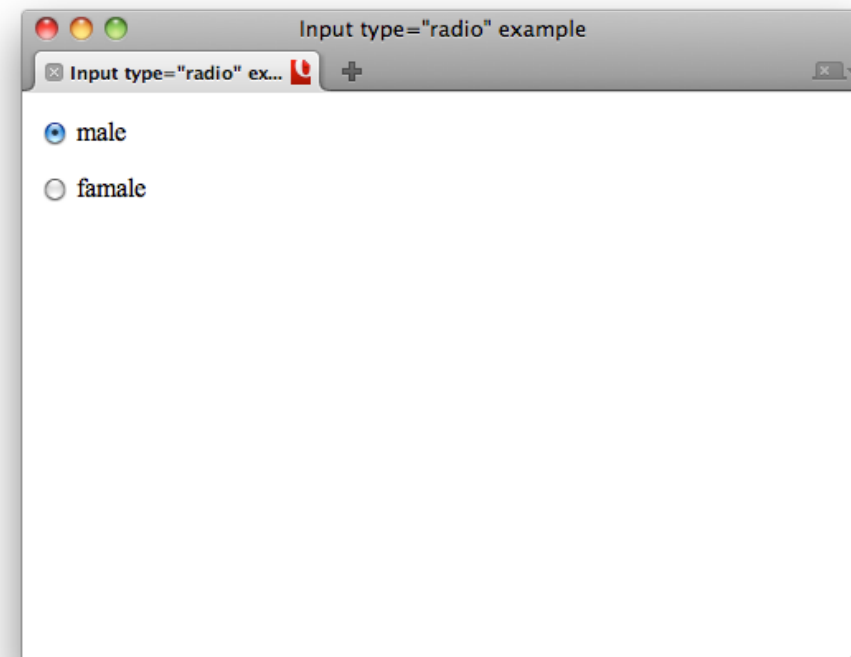


type="radio"

Crea un pulsante di opzione per selezione singola. A differenza delle checkbox, i radio button con lo stesso attributo name formano un gruppo mutuamente esclusivo: selezionarne uno deselecta automaticamente tutti gli altri del gruppo

- Il name condiviso crea il gruppo: il browser deselecta automaticamente gli altri
- Un radio non può essere deselectato dall'utente (solo cambiato)
- checked indica il valore preselezionato
- required su uno del gruppo rende obbligatoria la selezione

```
<label><input type="radio" name="male" checked> male</label>  
<label><input type="radio" name="famale"> famale</label>
```



type="textarea"

A differenza di `<input type="text">`, il `<textarea>` permette l'inserimento di testo su più righe. Ha un tag di chiusura e il suo contenuto iniziale (se presente) va scritto tra i tag, non in un attributo value. È ridimensionabile dall'utente di default (controllabile via CSS resize).

```
<form action="/form/submit" action="post">
<textarea name="comment" rows="12" cols="35">Send your comments to the author.</textarea>
<input type="submit" name="submitInfo" value="Submit">
</form>
```

rows	Numero di righe visibili (altezza iniziale). Non limita il testo inseribile. Default: 2.
cols	Numero di colonne visibili (larghezza). Meglio usare CSS width per il dimensionamento responsivo.
maxlength	Numero massimo di caratteri. Il browser impedisce fisicamente di digitare oltre il limite.
minlength	Numero minimo di caratteri. Validato al submit, non impedisce la digitazione.
placeholder	Testo suggerimento. Scompare alla digitazione. NON sostituisce <code><label></code> .
wrap	"soft" (default) = a capo visivo senza inserire \n; "hard" = inserisce \n dove va a capo (richiede cols).
readonly	Il testo è visibile e selezionabile ma non modificabile. Viene inviato con il form.
disabled	Il campo è disabilitato: non modificabile, non cliccabile, NON inviato con il form.

The screenshot shows a web browser window with the title "Title of the document" and the address bar displaying "https://www.w3docs.com". The main content area contains a text area with the placeholder text "Send your comments to the author." and a "Submit" button at the bottom.

type="email" e type="password"



type="email"

```
<input type="email" name="email"
      multiple required>
```

- Validazione automatica del formato email: il browser verifica la presenza di @ e un dominio valido prima dell'invio.
- Su mobile: tastiera ottimizzata con tasto @ visibile e dominio .com suggerito.
- L'attributo multiple permette di inserire più email separate da virgola.
- Validazione nativa non è perfetta: verifica solo la sintassi, non l'esistenza dell'indirizzo. Per sicurezza, validare sempre anche lato server.
- autocomplete="email" suggerisce indirizzi salvati.



type="password"

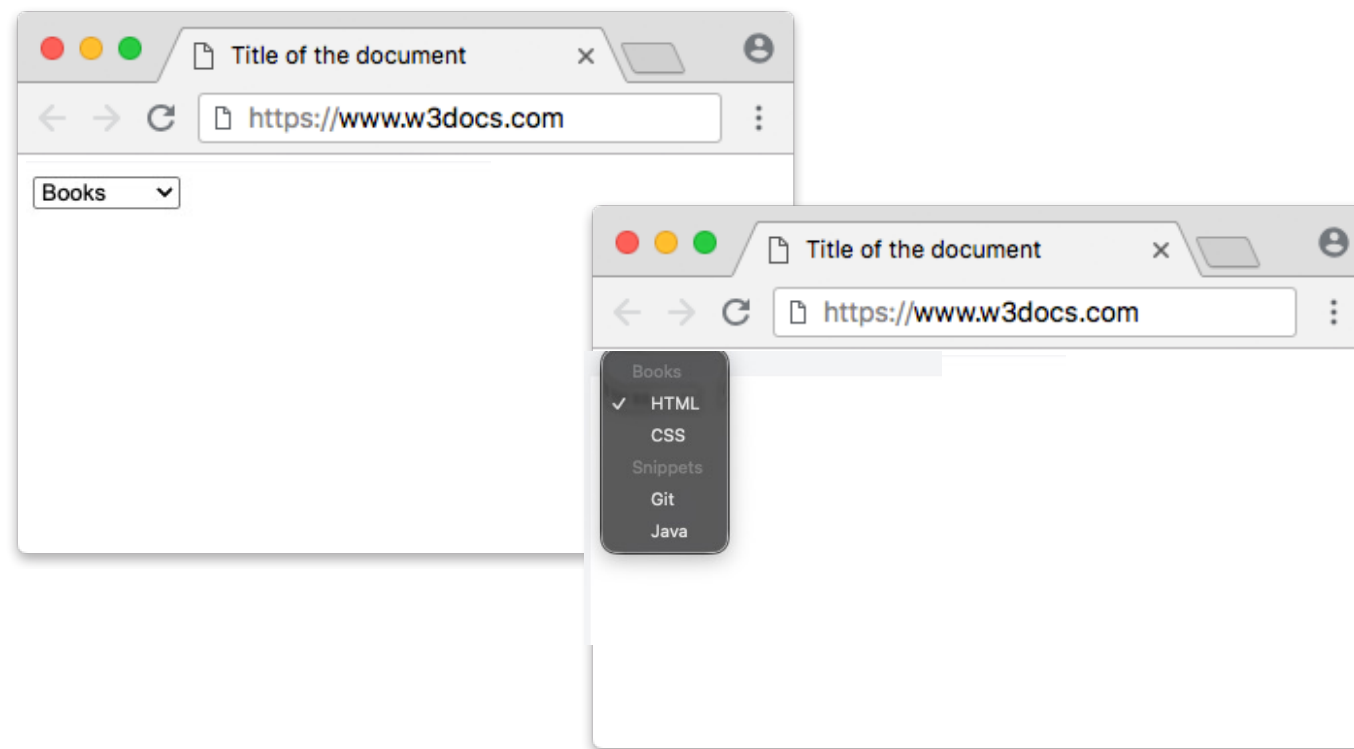
```
<input type="password" name="pwd"
      minlength="8" required>
```

- I caratteri digitati sono mascherati con pallini (•••) per impedire la lettura a chi è vicino all'utente.
- Il browser offre di salvare la password nel proprio password manager.
- Supporta minlength per lunghezza minima e pattern per requisiti di complessità
- autocomplete="new-password" per registrazione, "current-password" per login.
- Mai inviare password via GET! Usare sempre method="POST" e connessione HTTPS.

type="select"

L'utente sceglie da un elenco predefinito. `<optgroup>` raggruppa le opzioni con un'etichetta non selezionabile. La prima `<option>` con `value=""` funge da placeholder. L'attributo `selected` su un'option la preseleziona. `multiple` trasforma in lista a selezione multipla (usabilità scarsa).

```
<select aria-label="Books and Snippets">
  <optgroup label="Books">
    <option value="html">HTML</option>
    <option value="css">CSS</option>
  </optgroup>
  <optgroup label="Snippets">
    <option value="git">Git</option>
    <option value="java">Java</option>
  </optgroup>
</select>
```



type="number"

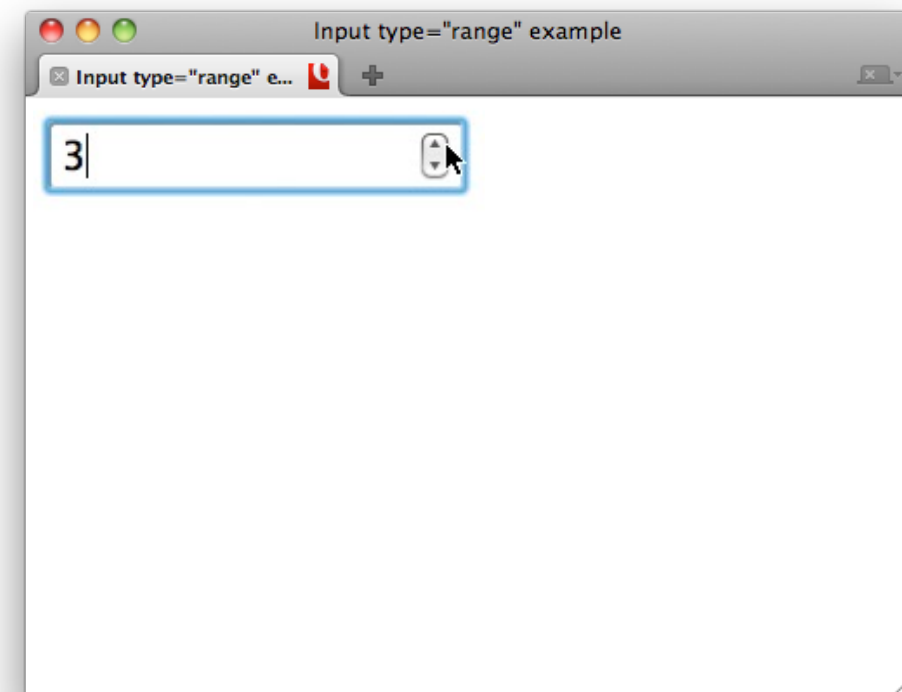
Campo numerico con frecce spinner (▲ ▼) per incrementare/decrementare il valore. Il browser impedisce l'inserimento di caratteri non numerici.

Attributi specifici:

- min — valore minimo accettato
- max — valore massimo accettato
- step — incremento di ogni click sullo spinner (default: 1).
- value — valore iniziale precompilato

La validazione verifica che il valore sia compreso tra min e max e sia multiplo di step.

```
<input type="number" min="0" max="100" value="0">
```



type="range"

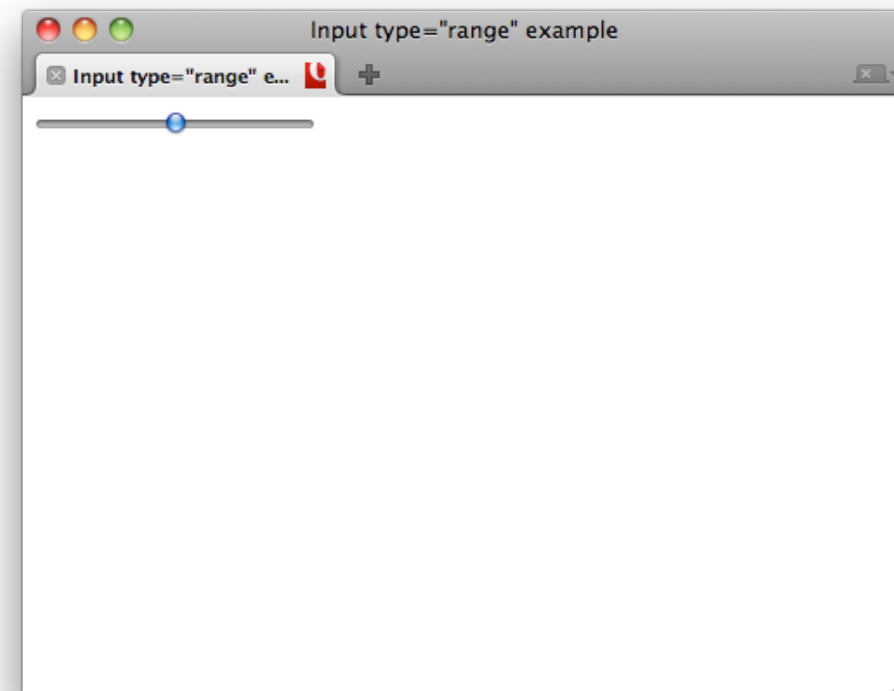
Slider orizzontale per selezionare un valore numerico in modo visuale. L'utente trascina un cursore tra il minimo e il massimo.

Differenze da number:

- Non mostra il valore numerico esatto.
- Non permette input diretto da tastiera (solo le frecce ← →)
- Ideale quando il valore esatto è meno importante dell'intervallo (volume, luminosità, opacità)

Stessi attributi: min, max, step, value.

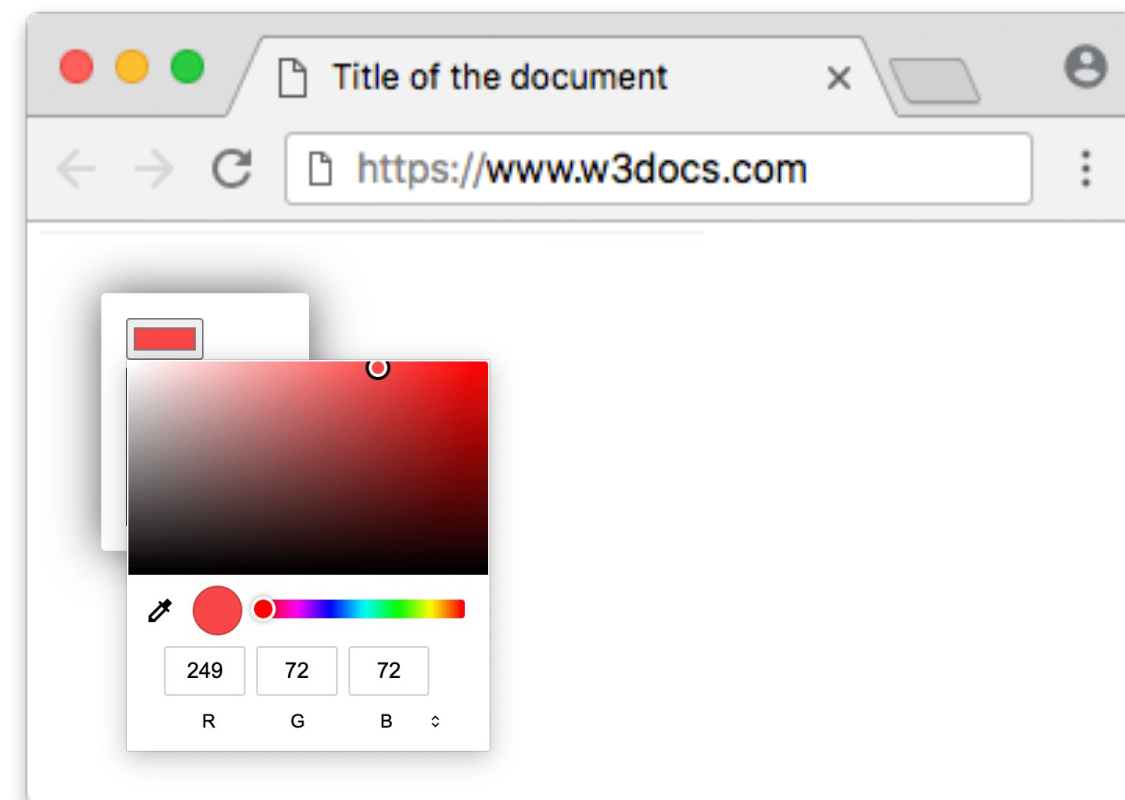
```
<input type="range" min="0" max="100" value="0" step="5">
```



type="color"

Apri il color picker nativo del sistema operativo, con ruota dei colori, palette, picker a occhio e input esadecimale. Il valore è sempre in formato #RRGGBB (6 cifre esadecimali). Non supporta alpha/trasparenza. Utile per personalizzazione temi, configuratori di prodotto, editor grafici. L'attributo value imposta il colore iniziale (default: #000000).

```
<input type="color" name="selectcolor">
```



type="date"

HTML5 offre selettori data/ora nativi del browser che eliminano la necessità di librerie JavaScript come DatePicker. Il formato del valore inviato è sempre standard (ISO 8601), indipendentemente dalla lingua dell'utente.

type="date" Selettore data con calendario nativo.

type="time" Selettore orario con ore e minuti.

type="datetime-local" Combinazione di data e ora in un unico campo.

type="week" Seleziona un mese e anno (senza giorno).

type="month" Seleziona una settimana dell'anno.

```
<input type="date" name="checkin-date">
```

Input Type	Input Field
date	<input type="date"/>
datetime	<input type="datetime" value="2011-01-17 00:05"/> UTC
datetime-local	<input type="datetime-local" value="2011-01-18 12:00"/>
month	<input type="month" value="2011-01"/>
week	<input type="week" value="2011-W03"/>
time	<input type="time" value="23:34"/>

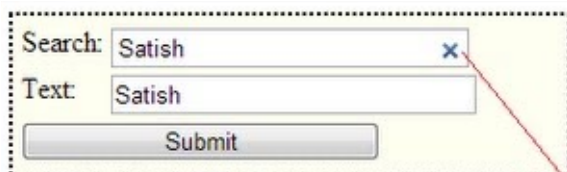
type="search", type="tel" e type="url"

type="search"

```
<input type="search" name="ser">
```

Campo ottimizzato per la ricerca. Alcuni browser mostrano un'icona di ricerca e un pulsante ✕ per cancellare rapidamente il testo.

Semanticamente identico a text.



Cancel

type="tel"

```
<input type="tel" name="telefono">
```

Su dispositivi mobile attiva la tastiera numerica con tasti per +, *, #.

NON ha validazione automatica integrata perché i formati telefonici variano troppo nel mondo (+39, 06/1234567, (212) 555-1234).

Phone number:

type="url"

```
<input type="url" name="sito">
```

Validazione automatica del formato URL: il browser verifica la presenza del protocollo (http:// o https://).

Accetta solo URL completi con protocollo: "example.com" viene rifiutato, "https://example.com" è valido. autocomplete="url" per suggerimenti.

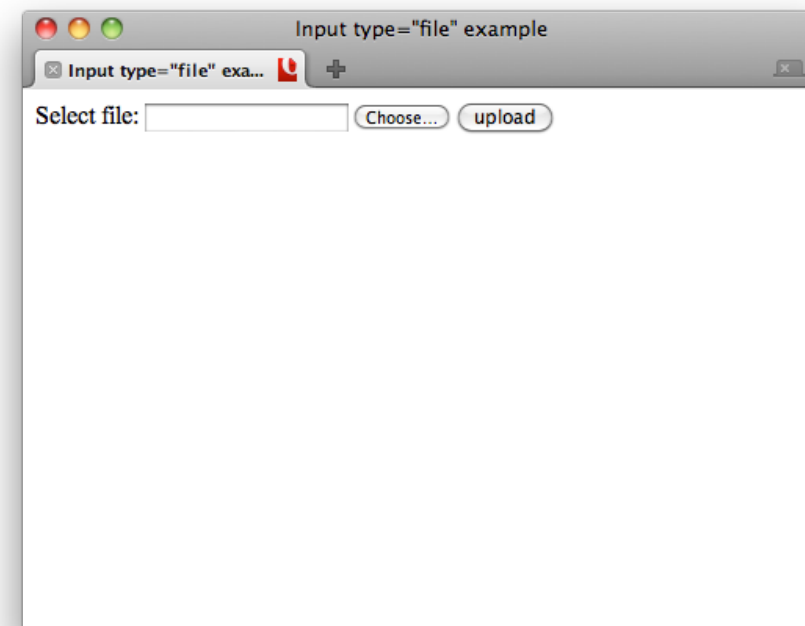
Website Name:

type="file"

crea un pulsante che apre il selettore di file del sistema operativo, permettendo all'utente di caricare uno o più file

- **accept** filtra i tipi di file mostrati nel selettore. Può contenere estensioni di file (.pdf, .jpg ...), tipi MIME (application/pdf, image/jpeg)
- **multiple** permette di selezionare più file contemporaneamente.

```
<input type="file" id="documento" name="curriculum"
accept=".pdf,.doc,.docx,application/pdf" required>
```



Organizzazione form

Gli elementi `<fieldset>` e `<legend>` raggruppano campi correlati con un bordo visivo e un'etichetta per il gruppo:

```
<form action="http://site.com/bin/adduser" method="post">
<fieldset>
<legend>Nome e cognome</legend>
Nome: <input type="text" name="nome"><br>
Cognome: <input type="text" name="cognome">
</fieldset>
<fieldset>
<legend>Provincia</legend>
<select name="provincia" multiple="multiple" size=7>
<optgroup label="Capoluogo">
<option value="BO" selected="selected">Bologna</option>
</optgroup>
<optgroup label="Emilia">
<option value="MO">Modena</option>
<option value="RE">Reggio Emilia</option>
<option value="PR">Parma</option>
<option value="PC">Piacenza</option>
</optgroup>
</select>
</fieldset>
</form>
```

Validazione

HTML5 offre un sistema di validazione integrato nel browser che funziona senza JavaScript. Il browser applica le regole automaticamente prima dell'invio del form, mostrando messaggi di errore localizzati nella lingua dell'utente.

required

```
<input required>
```

Il campo deve essere compilato. Il browser mostra "Compila questo campo" e impedisce l'invio. Funziona su text, email, password, select, textarea, checkbox, radio e file.

minlength / maxlength

```
<input minlength="3" maxlength="50">
```

Numero minimo/massimo di caratteri. maxlength impedisce fisicamente la digitazione oltre il limite; minlength mostra errore solo al submit.

min / max

```
<input type="number" min="1"
max="100">
```

Valore numerico o temporale minimo/massimo. Funziona su number, range, date, time, datetime-local, month, week.

step

```
<input type="number" step="0.01">
```

Incremento valido. Il valore deve essere multiplo di step (partendo da min o 0). step="any" accetta qualsiasi decimale.

pattern

```
<input pattern="[A-Z]{2}\d{5}">
```

Espressione regolare (regex) che il valore deve rispettare. Solo per input testuali (text, tel, email, url, password, search). Il pattern è ancorato automaticamente (^...\$).

type (implicita)

```
<input type="email">
```

Ogni tipo ha la sua validazione implicita: email verifica @, url verifica il protocollo, number accetta solo numeri, date solo date valide.

Riepilogo

Tipo	Descrizione	Tipo	Descrizione
text	Testo generico, campo default	email	Validazione @, tastiera con @
password	Caratteri mascherati (•••)	search	Icona ricerca, tasto ✕ per cancellare
tel	Tastiera numerica, no validazione	url	Validazione protocollo, tastiera /
number	Spinner ▲ ▼, min/max/step	range	Slider orizzontale min-max
date	Calendario nativo YYYY-MM-DD	time	Selettore orario HH:MM
datetime	Data + ora combinati	month	Mese e anno YYYY-MM
week	Settimana dell'anno YYYY-Wnn	checkbox	Selezione multipla indipendente
radio	Selezione singola nel gruppo name	color	Color picker nativo #RRGGBB
file	Upload file, accept, multiple	button	Pulsante generico gestito con JS
submit	Invia il form + validazione	reset	Riporta ai valori iniziali

Esempio

La Macchina del Caffè di StarBuzz

costruiremo insieme, un pezzo alla volta, il modulo d'ordine online di una caffetteria chiamata "StarBuzz Caffè".

```
<!DOCTYPE html>
<html lang="it">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>StarBuzz Caffè – Ordina Online</title>
  </head>
  <body>

  </body>
</html>
```

Esempio step 2

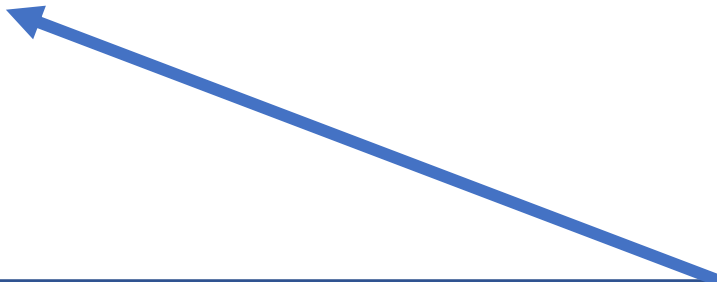
Ora aggiungiamo il contenuto visibile. All'interno del <body>

```
<body>  
  <h1>StarBuzz Caffè</h1>  
  <h2>La Macchina del Caffè</h2>  
  <p>Compila il modulo qui sotto e clicca "Ordina Adesso" per completare il tuo  
ordine.</p>  
</body>
```

Esempio step 3

Ora creiamo il form vero e proprio.

```
<form action="https://httpbin.org/post" method="post">  
</form>
```



httpbin.org è un servizio web gratuito e open source creato apposta per testare le richieste HTTP. Funziona come uno "specchio": riceve i dati che gli invii e te li restituisce in formato JSON, così puoi vedere esattamente cosa è stato trasmesso.

Esempio step 4

aggiungiamo il primo campo: la scelta del tipo di caffè. Utilizzeremo un menu a tendina <select>

```
<form action="https://httpbin.org/get" method="post">
  <label for="caffè">Scegli il tuo caffè:</label>
  <select id="caffè" name="caffè">
    <option value="miscela_casa">Miscela della Casa</option>
    <option value="bolivia">Bolivia Supremo</option>
    <option value="guatemala">Guatemala Biologico</option>
    <option value="kenya">Kenya AA</option>
  </select>
</form>
```


Esempio step 5

Nella pratica reale, è buona norma aggiungere un'opzione "segnaposto" che obblighi l'utente a fare una scelta consapevole. Modificate il <select> aggiungendo una prima opzione vuota:

```
<label for="caffè">Scegli il tuo caffè:</label>
<select id="caffè" name="caffè" required>
  <option value="">– Seleziona un caffè –</option>
  <option value="miscela_casa">Miscela della Casa</option>
  <option value="bolivia">Bolivia Supremo</option>
  <option value="guatemala">Guatemala Biologico</option>
  <option value="kenya">Kenya AA</option>
</select>
```

Esempio step 6

Dopo il <select>, aggiungiamo un gruppo di radio button per scegliere tra caffè in grani e caffè macinato.

```
<br><br>
```

```
<label>Tipo di macinatura:</label><br>
```

```
<input type="radio" id="grani" name="macinatura" value="grani">
```

```
<label for="grani">In grani</label>
```

```
<input type="radio" id="macinato" name="macinatura" value="macinato" checked>
```

```
<label for="macinato">Macinato</label>
```

Esempio step 7

Aggiungiamo un campo per scegliere quante confezioni ordinare:

```
<br><br>
```

```
<label for="quantita">Numero di confezioni:</label>
```

```
<input type="number" id="quantita" name="quantita" min="1" max="10" value="1">
```

Esempio step 8

Aggiungiamo un campo per la data di consegna desiderata:

```
<br>
<label for="data_consegna">Data di consegna desiderata:</label>
<input type="date" id="data_consegna" name="data_consegna">
```

Esempio step 9

Aggiungiamo delle caselle di controllo per opzioni aggiuntive:

```
<br><br>
```

```
<label>Extra:</label><br>
```

```
<input type="checkbox" id="confezione_regalo" name="extra[]" value="confezione_regalo">
```

```
<label for="confezione_regalo">Confezione regalo</label>
```

```
<br>
```

```
<input type="checkbox" id="catalogo" name="extra[]" value="catalogo" checked>
```

```
<label for="catalogo">Includi il catalogo nell'ordine</label>
```

```
<br>
```

```
<input type="checkbox" id="campione" name="extra[]" value="campione">
```

```
<label for="campione">Aggiungi campione gratuito</label>
```

Esempio step 10

Fino a qui abbiamo messo tutti i campi uno dopo l'altro. Il nostro form funziona, ma non è ben organizzato. Ora raggruppiamo i campi in due sezioni logiche utilizzando `<fieldset>` e `<legend>`.

```
<form action="https://httpbin.org/get" method="GET">
  <fieldset>
    <legend>Dettagli dell'ordine</legend>

    <!-- Qui dentro mettete TUTTO il codice dei passi 4-9 -->
    <!-- Select del caffè, radio della macinatura, -->
    <!-- campo quantità, campo data, checkbox extra -->

  </fieldset>
</form>
```

Esempio step 11

Ora aggiungiamo un secondo fieldset per i dati di spedizione. Scrivete questo codice dopo la chiusura `</fieldset>` del primo gruppo, ma sempre dentro il `<form>`:

```
<fieldset>
  <legend>Dati di spedizione</legend>

  <label for="nome">Nome e cognome:</label><br>
  <input type="text" id="nome" name="nome" placeholder="es. Mario Rossi" required>
  <br><br>

  <label for="indirizzo">Indirizzo:</label><br>
  <input type="text" id="indirizzo" name="indirizzo" placeholder="es. Via Roma 1" required>
  <br><br>

  <label for="citta">Città:</label><br>
  <input type="text" id="citta" name="citta" placeholder="es. Roma" required>

</fieldset>
```

Esempio step 12

Aggiungiamo altri due campi dentro il fieldset "Dati di spedizione", dopo il campo città

```
<br><br>
<label for="telefono">Telefono:</label><br>
<input type="tel" id="telefono" name="telefono"
  placeholder="es. 06 1234567"
  pattern="[0-9\s\+\-]{7,15}"
  title="Inserisci un numero di telefono valido"
  required>
<br><br>

<label for="email">Email:</label><br>
<input type="email" id="email" name="email"
  placeholder="es. mario.rossi@email.it"
  required>
```


Esempio step 13

Aggiungiamo un'area di testo per commenti aggiuntivi, sempre dentro il fieldset "Dati di spedizione":

```
<br><br>
<label for="commenti">Note aggiuntive:</label><br>
<textarea id="commenti" name="commenti" rows="4" cols="40"
  placeholder="Istruzioni speciali per la consegna, preferenze..."
  maxlength="500"></textarea>
```

Esempio step 14

Ora aggiungiamo il pulsante che permette di inviare il form. Scrivete questo codice dopo il `</textarea>`, ma ancora dentro il secondo `<fieldset>`:

```
<br><br>  
<button type="submit"> Ordina Adesso</button>
```

Dopo il pulsante di invio, aggiungiamo un pulsante per cancellare il form:

```
<button type="reset">Cancella tutto</button>
```

Dettagli dell'ordine

Scegli il tuo caffè:

— Seleziona un caffè — ▾

Tipo di macinatura:

☐ In grani

☒ Macinato

Numero di confezioni:

1

Data di consegna desiderata:

gg/mm/aaaa

Extra:

☐ Confezione regalo

☒ Includi il catalogo nell'ordine

☐ Aggiungi campione gratuito

Dati di spedizione

Nome e cognome:

es. Mario Rossi

Indirizzo:

es. Via Roma 1

Città:

es. Roma

Telefono:

es. 06 1234567

Email:

es. mario.rossi@email.it

Note aggiuntive:

Istruzioni speciali per la consegna, preferenze...

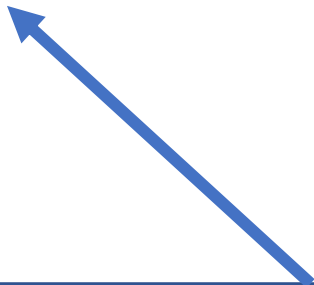
Ordina Adesso

Cancella tutto

Esempio step 15

Ora creiamo il form vero e proprio.

```
<form action="https://httpbin.org/get" method="get">  
</form>
```



Cosa succede se usiamo la GET?

Digitato

StarBuzz Caffè

La Macchina del Caffè

Compila il modulo qui sotto e clicca "Ordina Adesso" per completare il tuo ordine.

Dettagli dell'ordine

Scegli il tuo caffè: — Seleziona un caffè —

Tipo di macinatura:
☐ In grani ☒ Macinato

Numero di confezioni:

Data di consegna desiderata:

Extra:
☐ Confezione regalo

Città:

Telefono:

Email:

Note aggiuntive:

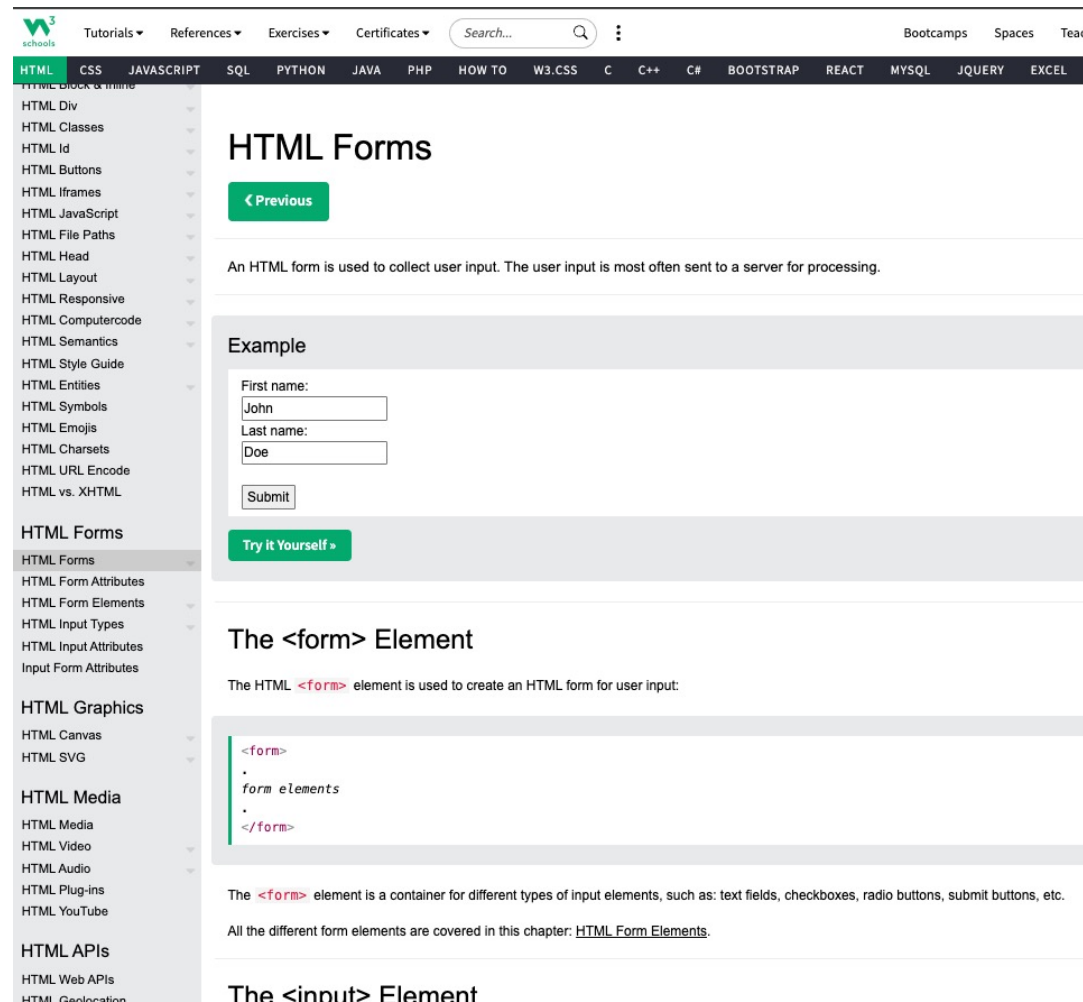
Ordina Adesso

Cancella tutto

```
https://httpbin.org/get?caffe=bolivia&macinatura=grani&quantita=2&data_consegna=2026-03-13&extra%5B%5D=confezione_regalo&extra%5B%5D=catalogo&nome=Attilio&indirizzo=via+fasulla&citta=salerno&telefono=333333333&email=adellagreca%40unisa.it&commenti=
```

References

Guida ai Form in HTML: https://www.w3schools.com/html/html_forms.asp



The screenshot shows the W3Schools website with the 'HTML Forms' page selected. The left sidebar contains a navigation menu with categories like HTML, CSS, JavaScript, SQL, Python, Java, PHP, and more. The main content area is titled 'HTML Forms' and includes a 'Previous' button, an introductory paragraph, an 'Example' section with a form, a 'Try It Yourself' button, and sections for 'The <form> Element' and 'The <input> Element'.

HTML Forms

< Previous

An HTML form is used to collect user input. The user input is most often sent to a server for processing.

Example

First name:

 Last name:

Try It Yourself »

The <form> Element

The HTML `<form>` element is used to create an HTML form for user input:

```
<form>
  .
  form elements
  .
</form>
```

The `<form>` element is a container for different types of input elements, such as: text fields, checkboxes, radio buttons, submit buttons, etc.

All the different form elements are covered in this chapter: [HTML Form Elements](#).

The <input> Element